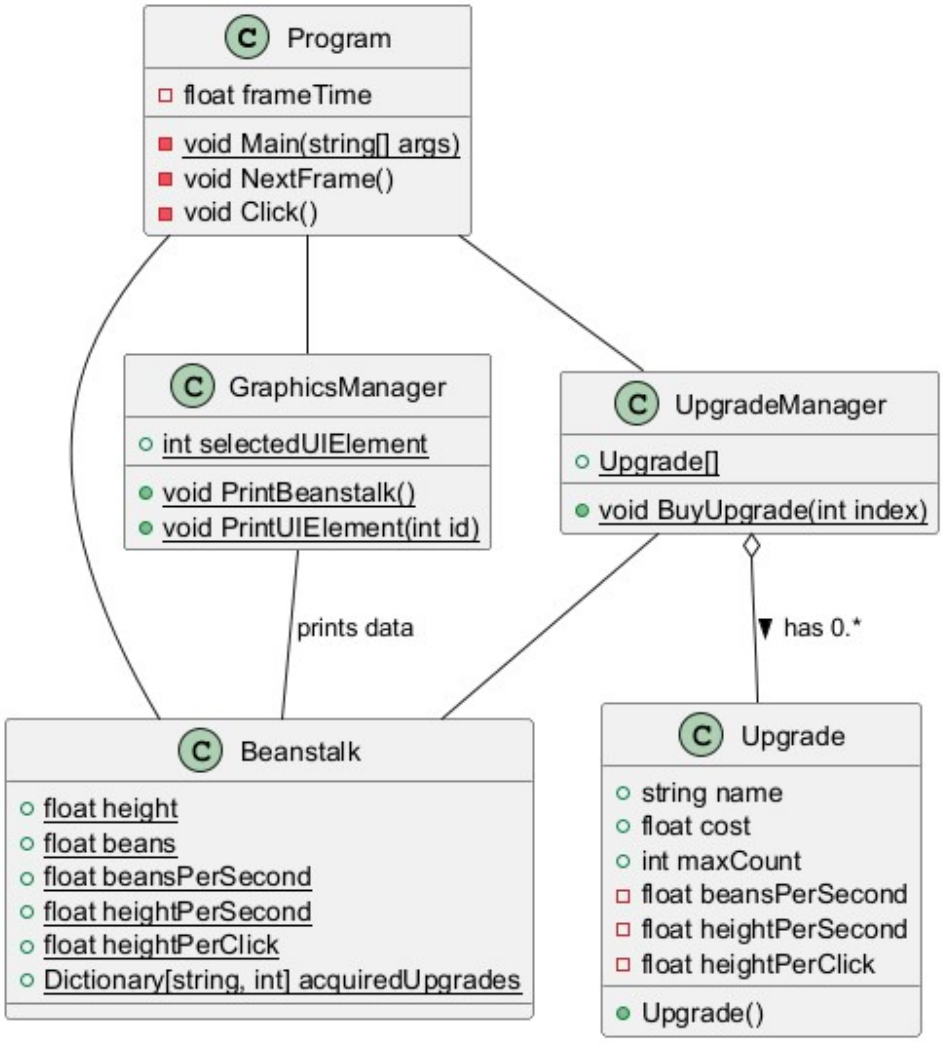
GIT https://github.com/Murkorus/ProjektOOA

Mandag Brainstorm

Vi starter med at spillet er et idle spil, som spilles i konsollen. Det kunne have noget med planter at gøre, planter vokser, og idle spil handler om at få ting til abstrakt set at vokse.
Man kunne måske bruge en timer af en slags til at se hvor lang tid der er gået mellem hver play session.
Planter vækker minder om Plants vs Zombies, måske kunne spillet omhandle et lignende koncept enten med at man gror planter for at beskytte et eller andet, eller at tage inspiration fra Zen Garden.
Man kunne også tage mere inspiration fra Cookie Clicker, og have bare én enkelt plante man gror større og større. Evt. Kunne det være en beanstalk, ligesom i eventyret, som man gror og får score af at vokse den uendeligt.
Planten/planterne man vokser kunne gro bær som man er måden man tjener penge på. Mayhaps magic beans.
Værktøjer spilleren kan bruge til at passe planten.
Evt. få denne her beanstalk til at gro ud ad konsolvinduet så man skal scrolle op for at se den hele.
Bønnestagen kunne vokse automatisk højere så længe bestemte stats (lys, vand, etc.) er over et bestemt niveau.
De her stats kan blive vedligeholdt ved at spilleren lave et input af en slags, så vi har noget i koden at respondere til. Med andre ord, så bliver spillet hændelsesbaseret.
Siden vi igennem konsollen ikke kan tilgå tekst der er blevet printet og skubbet ud af det aktive vindue, vil vi bare følge toppen af bønnestagen så vi kan lave sejt ascii art når den bliver højere og højere.

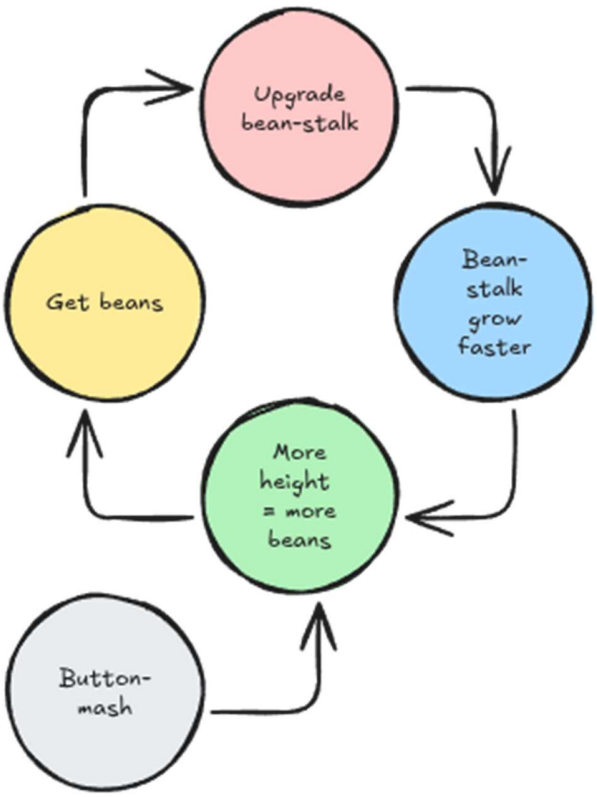
Klasse diagram

Vi bruger et program der hedder PlantUML som opsætter diagrammet ud fra kode vi skriver.



Gameplay loop

Gameplay loop inspireret af Cookie Clicker, fordi vi gerne vil holde konceptet simpelt. Spilleren trykker på en tastaturknap for at få bønnestagen til at vokse. Spilleren får bønner passivt baseret på hvor høj bønnestagen er blevet. Bønner kan bruges til at købe opgraderinger som lader bønnestagen vokse hurtigere.


Upgrade bean-stalk
Bean-stalk grow faster
More height = more beans
Button-mash
Get beans

Logbog

Mandag

Vi (Laurids og Tjalfe) startede arbejde klokken 09:45. På dette tisdpunkt har vi allerede fra før ferien blevet enige om et vagt koncept. Vi vil lave vores spil som en konsol applikation, og vi vil lave et idle spil, fordi et idle spil er relativt simpelt at lave, og bliver ikke markant forværret af mangel på graphics. Vi begyndte at brainstorme videre på hvad spillet skulle indeholde.

Upgrades tænkte vi først som sub-classes, men bestemte at det ville være simplere at have hver upgrade fungere som et objekt fra en enkelt class.

Vi har fået lavet en UML og imorgen vil vi forsøge at implementere det i C#, og se hvilke problemer dukker op.

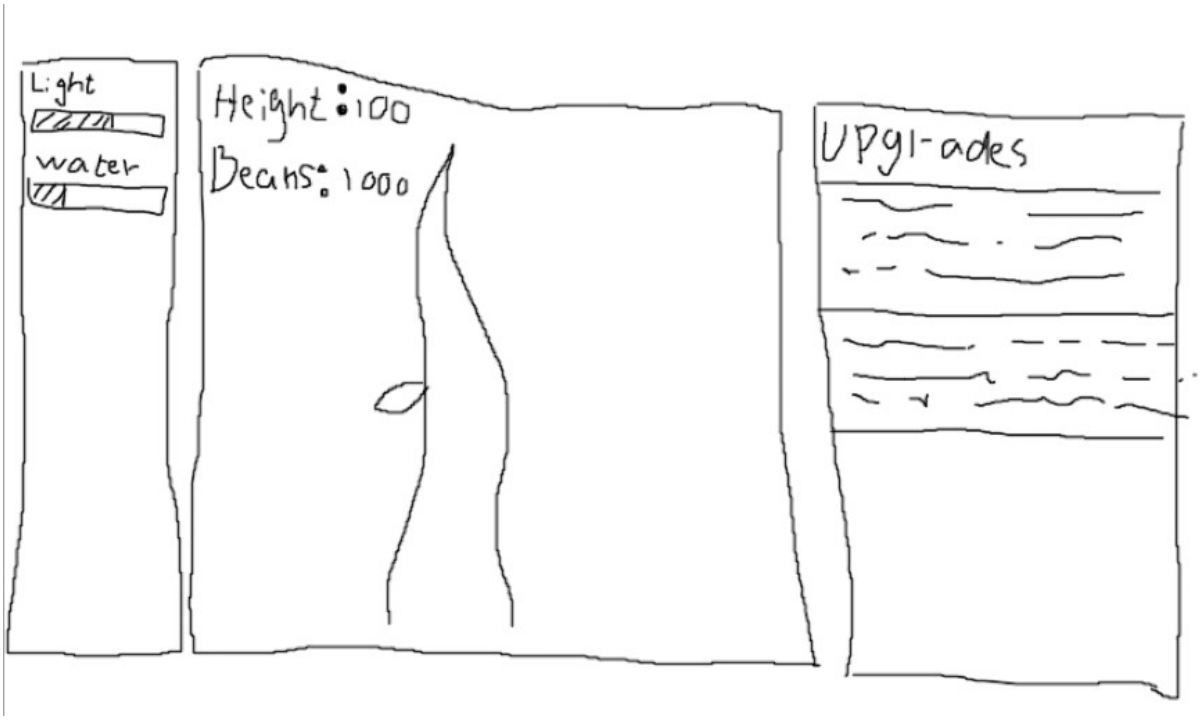
Tirsdag

Idag vil vi implementere funktionelitet ud fra vores diagram og arbejde.

TODO:

- Laurids
    - Bean grafik der kan opdateres.
        - Prøvede at bruge Ascii specifike filer men kom frem til det nok var nemmere og hurtigere bare at have en .cs fil med de ascii sprites vi skulle bruge.
        - Ok whatever! Ascii bliver bare opbevaret som variabler i selve GraphicManager'en.
        - Jeg har opdateret koden i GraphicsManager så den kan loade eksterne filer. Du kan putte dine ascii art .txt filer ind i mappen "OOAGame > bin > Debug > net10.0 > Art", navngivet "Art0.txt", "Art1.txt", "Art2.txt", osv. :) -Tjalfe
- Tjalfe
    - "Upgrade Shop" funktionelitet, 3 upgrades
        - Har lavet 3 upgrades der ændrer på lidt forskellige ting. Sprinkler, Gardener, og Fertilizer. Siden vores spil er baseret på at få en bønnestage til at gro, tænker jeg det giver mening at vores opgraderingsnavne er relateret til planter på en eller anden måde.
        - Har sat metoder op som tillader nemt og overskueligt udvidet funktionalitet i fremtiden.

- Bean math equations.
    - Lige nu er funktionen bare lineær. Spilleren får 0.01 bønner per sekund per bønnestage height. Så f(x)=0.01 * x, hvor x er height, og y er bønner per sekund. Jeg tror det ville være bedre at gøre funktionen eksponentiel, eller i hvert fald til et polynomium af en slags. Priserne på opgraderinger skalerer lige nu eksponentielt med 15% per køb (baseCost * 1.15^n, hvor n er mængden af gange man har købt en opgradering). Siden priser stiger eksponentielt, ville det også give mening at produktion stiger eksponentielt, men de skal ikke følges ad præcist. Evt. kan man gøre så funktionsparametrene eller endda selve funktionen udskiftes baseret på opgraderinger?
- Set up user interface
    - Har imported en metode til at tegne en boks fra vores første spilprojekt, lavet af Mikkel, men har ændret den lidt så den passer til vores projekt. Jeg vil gerne bruge denne metode til at opdele skærmen så man har en boks hvor bønnestagen bliver vist, og en boks hvor der er opgraderinger spilleren kan vælge imellem, og evt. flere bokse rundt omkring hvis vi kommer med nogle idéer. Spilleren skal så kunne navigere rundt imellem relevante UI-elementer (f.eks. vælge opgraderinger) ved at bruge WASD eller piletasterne. Har lavet et scuffed lille mockup af hvordan jeg forestiller mig det vil se ud:



    - Er ikke kommet så langt fordi konsollen virker anderledes mellem Windows 10 og Windows 11. Før jeg kan definere hvordan UI'et skal set ud, vil jeg gerne vide præcist hvor stor konsol vinduet er, og gerne kunne manuelt ændre størrelsen. Jeg ved ikke om det er muligt på Windows 11, fordi Console.SetWindowSize() ikke virker i den nye terminal. Det kan godt være vi bliver nødt til at bruge default console size (120x30), og bare håbe på at brugeren ikke har ændret det.

Jeg føler ikke lige for at opdatere vores klassediagram endnu, siden det alligevel bliver ændret igen næsten med det samme. Det følger stadig samme struktur, jeg har bare ændret lidt på hvordan det var ment at de forskellige klasser skulle snakke sammen. Jeg har også gjort de fleste variabler public, fordi der sker ikke rigtigt noget ved at de bliver ændret udefra. De få ting der godt kan tage skade af at blive ændret skal jeg nok gøre private lidt senere, men jeg vil lige få lavet spillet funktionelt før jeg tænker på god kode practice :p

- Tjalfe

Onsdag

Vi skal bruge X antal tegninger af bønnestagen i den forskellige stadier. Dimensionen er 78x28. Vi skaber disse tegninger med bekendte redskaber. Vi laver det først som pixel-art i Aseprite, og så smider det ind i en Pixel-Art til Ascii converter. -laurids

Jeg har lavet et script der kan slice et 'spritesheet' af ascii til flere tekst filer.

TODO

- Implementer grafik på bønnestage

Der var et problem med GitHub, vores .gitignore fil ignorerede det directory som vores ascii art filer skal ligge i, jeg har fikset det nu. – Tjalfe

Jeg nåede ikke at blive helt færdig med at lave UI i går, men det er basically færdigt nu. Jeg mangler at skrive nogle comments her og der, men jeg vil heller ikke bruge for lang tid på det lige nu, så du kan få adgang til scriptet og lave bønnestage grafik. – Tjalfe

Jeg vil gå i gang med at gøre lidt rent i koden så det er mere sikkert og læsbart, samt skrive nogle kommentarer i nogle af de andre scripts. Efter det, hvis jeg bliver færdig i dag, kan jeg opdatere vores klassediagram. Skal stadig lige have fundet ud af hvad aggregation og composition og alle de der ord betyder. – Tjalfe

Paths i en IDE er lidt forvirrende, da programmet ikke nødvendigvis kører fra det samme sted som dit projekt. - Laurids

Torsdag

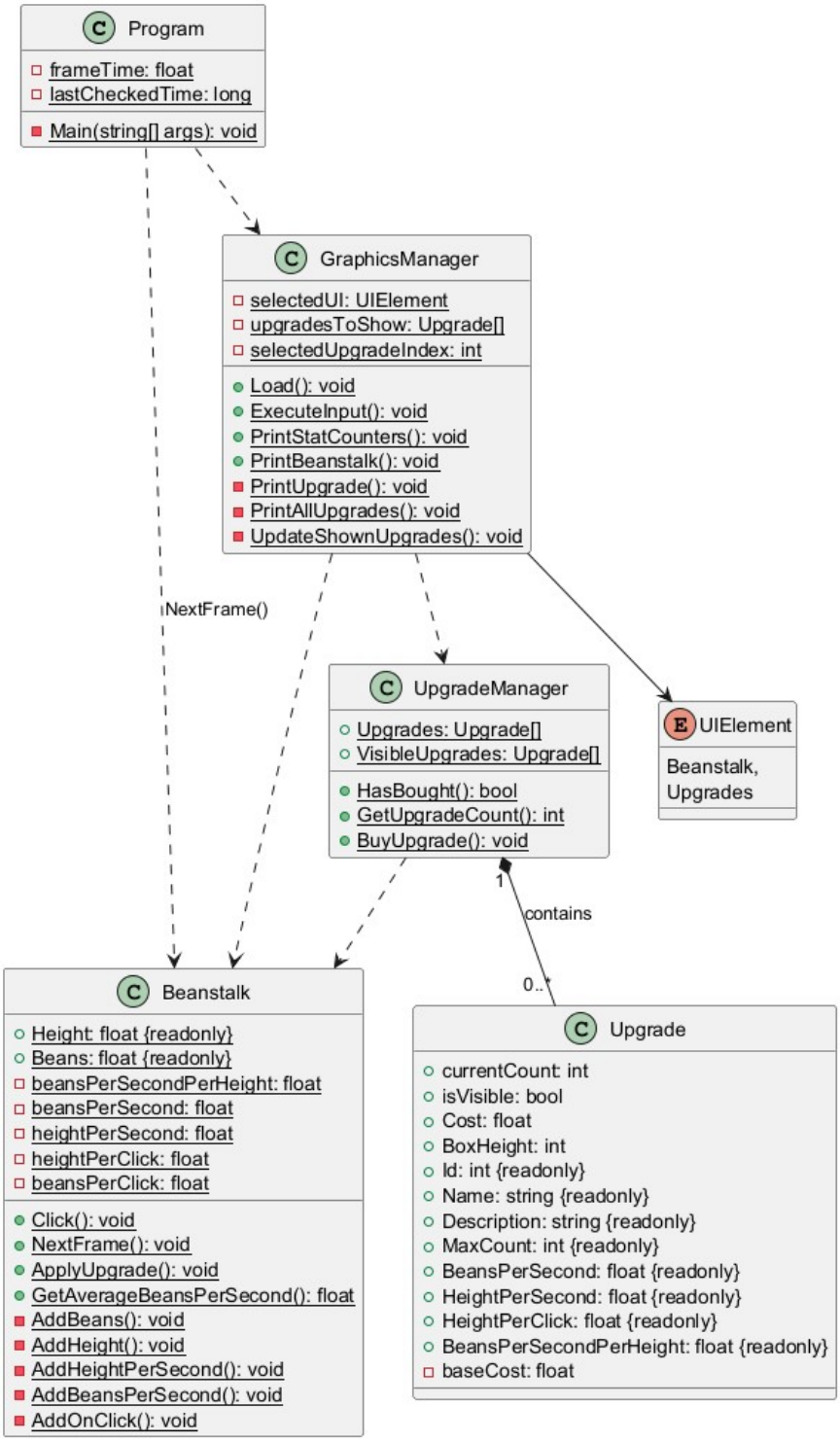
Jeg nåede at få opdateret koden i går til at være bedre struktureret, så i dag vil jeg opdatere vores klassediagram. Lige nu mangler vi ikke rigtigt at skrive noget kode. Spillet virker. Det eneste vi mangler, er at implementere bønnestagegrafikken, og måske at lave nogle flere opgraderinger, lige nu har vi nemlig stadig kun 3. – Tjalfe

Jeg har opdateret vores klassediagram nu, så det reflekterer den overordnede kodestruktur. – Tjalfe

Jeg har brugt det system Tjalfe har designet til at implementere en “Sell Trimmings” upgrade. Samt implementeret vores nye bønnestok grafik, med et system der dynamisk opdaterer den størelse. -laurids

Har også tilføjet 4 små opgraderinger.

Oh shit oh fuck vi har ikke gjort brug at nedarvning og interfaces!
Note til næste gang: “læs opgaven”


Program
frameTime: float
lastCheckedTime: long
Main(string[] args): void
GraphicsManager
selectedUI: UIElement
upgradesToShow: Upgrade[]
selectedUpgradeIndex: int
Load(): void
ExecuteInput(): void
PrintStatCounters(): void
PrintBeanstalk(): void
PrintUpgrade(): void
PrintAllUpgrades(): void
UpdateShownUpgrades(): void
NextFrame()
UpgradeManager
Upgrades: Upgrade[]
VisibleUpgrades: Upgrade[]
HasBought(): bool
GetUpgradeCount(): int
BuyUpgrade(): void
UIElement
Beanstalk,
Upgrades
1
contains
0..*
Beanstalk
Height: float {readonly}
Beans: float {readonly}
beansPerSecondPerHeight: float
beansPerSecond: float
heightPerSecond: float
heightPerClick: float
beansPerClick: float
Click(): void
NextFrame(): void
ApplyUpgrade(): void
GetAverageBeansPerSecond(): float
AddBeans(): void
AddHeight(): void
AddHeightPerSecond(): void
AddBeansPerSecond(): void
AddOnClick(): void
Upgrade
currentCount: int
isVisible: bool
Cost: float
BoxHeight: int
Id: int {readonly}
Name: string {readonly}
Description: string {readonly}
MaxCount: int {readonly}
BeansPerSecond: float {readonly}
HeightPerSecond: float {readonly}
HeightPerClick: float {readonly}
BeansPerSecondPerHeight: float {readonly}
baseCost: float

Fredag

Use Case Section	Comment
Name	Klik
Description	Spilleren trykker på en knap (spacebar) for at få resurser i spillet ved bl.a. at gro bønnestagen.
Actors	Spilleren, bønnestagen
Preconditions	Spilleren skal have bønnestagen valgt som det UI element de kigger på.
Main Success Scenario	Bønnestagen bliver højere, og spilleren får andre resurser hvis det er relevant fra f.eks. opgraderinger.
Alternative Scenario	Hvis spilleren har klikket for hurtigt vil inputtet ikke blive behandlet, så der er et cooldown på hvor hurtigt man kan klikke.
Miscellaneous	

Use Case Section	Comment
Name	Køb Upgrade
Description	Spilleren kan købe upgraderinger, der hjælper til at få flere points.
Actors	Spilleren, bønnestagen
Preconditions	Spilleren skal have nok 'beans' til at have råd til upgraden
Main Success Scenario	Spilleren har nok 'beans' og køber upgraden.
Alternative Scenario	Spilleren har ikke nok 'beans' og kan dermed ikke få upgraden.
Miscellaneous	

Use Case Section	Comment
Name	Navigér
Description	Spilleren kan navigere op, ned, højre og venstre i menu'en
Actors	Spilleren
Preconditions	Der er UI elementer i den pågældende retning.
Main Success Scenario	UI fokus bevæger sig
Alternative Scenario	UI fokus bevæger sig ikke, da der ikke er noget element den retning.
Miscellaneous	